Data Analysis Programming
Semester 2026-I
Practice I — Apache Airflow and PostgreSQL with Docker Compose

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the first guided practice of the *Data Analysis Programming* course!

This practice introduces the foundational infrastructure that will support the course project throughout the semester. Students will bring up a `local data engineering environment` using **Docker Compose**, running both **Apache Airflow** and **PostgreSQL** as containerized *services*. A provided example `DAG` will demonstrate how Airflow connects to **PostgreSQL** through a managed `SQL connection`, and students will verify the database state independently using **DBeaver**.

The goal is not to build anything from scratch yet — it is to *understand the environment*, get comfortable navigating the **Airflow UI**, and confirm that all components are communicating correctly before the project work begins.

General Practice Definition: Environment setup and connection verification involve deploying a `multi-service infrastructure` locally using **Docker Compose**, configuring inter-service communication through **Airflow Connections**, executing a pre-built `DAG` that queries **PostgreSQL** via a `PostgresHook`, and validating the database state from an external `SQL client`. These skills are foundational to every subsequent workshop in the course.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Tools and Resources

- **Apache Airflow:** Open-source workflow orchestration platform. Official documentation and Docker Compose quick-start: `https://airflow.apache.org/docs/`
- **Airflow Docker Compose quick-start (official):** `https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html`
- **DBeaver Community Edition (download):** Free, open-source universal database client. Download at: `https://dbeaver.io/download/`
- **DBeaver documentation and PostgreSQL connection guide:** `https://dbeaver.com/docs/dbeaver/`
- **PostgreSQL official documentation:** `https://www.postgresql.org/docs/`
- **Docker and Docker Compose:** `https://docs.docker.com/compose/`

Practice Scope and Objectives

- **Environment Deployment:** Bring up a fully functional local stack composed of `Apache Airflow` and `PostgreSQL` using the provided `Docker Compose` configuration.
- **Airflow Connection Configuration:** Create and validate an *Airflow Connection* of type *Postgres* that the example *DAG* uses to reach the database.
- **DAG Execution and Verification:** Trigger the provided example *DAG*, inspect task *logs*, and confirm that the `PostgreSQL` query executes successfully and returns the expected output.
- **External Database Inspection:** Connect to the `PostgreSQL` instance from **DBeaver** using the published *host port*, run a verification query, and browse the *database schema*.

Step-by-Step Guide

1. **Folder Setup:**
   - Create a working folder for this practice, for example `practice_1/`
   - Inside it, create a `dags/` subfolder — this is the folder that will be mounted into the Airflow container and where Airflow will discover DAG files
   - Copy the provided `docker-compose.yml` and `.env` files into the root of the working folder
   - Copy the provided example DAG file into the `dags/` subfolder
2. **Starting the Stack:**
   - Open a terminal in the working folder and run: `docker compose up -d`
   - Docker will pull the required images (this may take a few minutes on the first run) and start both the `postgres` and `airflow` services
   - Verify both containers are running: `docker compose ps`
   - Check Airflow logs if the webserver takes longer than expected: `docker compose logs -f airflow`
   - Once started, open the Airflow UI in a browser at: `http://localhost:8080` (credentials: `admin` / `admin`)
3. **Creating the Airflow Connection to PostgreSQL:**
   - In the Airflow UI, navigate to **Admin** → **Connections**
   - Click the + button to create a new connection and fill in the following fields:
     - *Conn Id:* `postgres_local`
     - *Conn Type:* `Postgres`
     - *Host:* `postgres` (the service name inside the Docker Compose network)
     - *Schema:* `airflow`
     - *Login:* `airflow`
     - *Password:* `airflow`
     - *Port:* `5432`
   - Click **Save**
   - Note: the host `postgres` is only valid from within the Docker network (i.e., from Airflow tasks). When connecting from your laptop (e.g., DBeaver), use `localhost` and the published port instead
4. **Triggering and Verifying the Example DAG:**
   - In the Airflow UI DAGs list, locate `example_postgres_check`
   - Enable the DAG using the toggle switch on the left, then click the **Trigger DAG** button (play icon)

- Once the run completes, click on the DAG run, then click on the `check_postgres` task, and open the **Logs** tab
- Confirm the logs contain a line similar to: `Postgres version: [('PostgreSQL 13.x ...',)]`
- A successful log output confirms that Airflow is correctly reaching PostgreSQL through the managed connection

5. **Connecting to PostgreSQL from DBeaver:**
   - Download and install DBeaver Community Edition from: `https://dbeaver.io/download/`
   - Open DBeaver and go to **Database** → **New Database Connection**
   - Select **PostgreSQL** from the list and click **Next**
   - Enter the following connection parameters:
     - *Host:* `localhost`
     - *Port:* `5432`
     - *Database:* `airflow`
     - *Username:* `airflow`
     - *Password:* `airflow`
   - Click **Test Connection** to verify connectivity, then click **Finish**
   - In the Database Navigator panel, expand the connection to browse schemas and tables
   - Open a SQL editor (**SQL Editor** → **Open SQL Script**) and run: `SELECT version();`
   - Confirm the query returns the PostgreSQL version string

Provided Docker Compose Configuration

The following `docker-compose.yml` is the one provided for this practice. It defines two services: `postgres` (the database) and `airflow` (combining the webserver and scheduler in a single container, which is sufficient for practice purposes). The `dags/` folder on the host is mounted into the container so that Airflow detects DAG files automatically.

```
version: '3.7'

services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  airflow:
    image: apache/airflow:2.7.1
    depends_on:
      - postgres
    environment:
      AIRFLOW__CORE__LOAD_EXAMPLES: "False"
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: >-
        postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
    ports:
      - "8080:8080"
    volumes:
      - ./dags:/opt/airflow/dags
    entrypoint: >
      bash -c "airflow db upgrade &&
               airflow users create
                 --username admin --firstname Air --lastname Flow
                 --role Admin --email admin@example.com
                 --password admin || true &&
               airflow webserver & airflow scheduler"

volumes:
  pgdata:
```

Provided Example DAG

Save the following file as `dags/example_postgres_dag.py`. It defines a single-task DAG that uses Airflow's `PostgresHook` (the managed SQL connector) to run a simple `SELECT version()` query against PostgreSQL and print the result to the task log.

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook

with DAG(
    dag_id='example_postgres_check',
    start_date=datetime(2024, 1, 1),
    schedule_interval=None,
    catchup=False
) as dag:

    def check_postgres():
        hook = PostgresHook(postgres_conn_id='postgres_local')
        records = hook.get_records('SELECT version();')
        print('Postgres version:', records)

    t1 = PythonOperator(
        task_id='check_postgres',
        python_callable=check_postgres
    )

    t1
```

The `PostgresHook` resolves the connection parameters (host, port, credentials, schema) from the Airflow Connection with `Conn Id = postgres_local`, created in Step 3 above. This is the recommended approach in production pipelines — credentials are never hard-coded in DAG source code.

Troubleshooting

- **Airflow container fails to start:** Check logs with `docker compose logs -f airflow`. A common cause on Linux is a file permission issue with the `dags/` volume; set the `AIRFLOW_UID` variable in a `.env` file to your user UID (`echo $(id -u)` and write `AIRFLOW_UID=<result>` in the `.env` file).

- **DAG task fails with connection error:** Verify the Conn Id in the DAG (`postgres_local`) exactly matches the one created in Admin → Connections. Inside Docker the host must be `postgres`, not `localhost`.

- **Missing provider package:** If Airflow raises an `ImportError` for `airflow.providers.postgres`, the Postgres provider is not bundled in the image. Install it by adding `apache-airflow-providers-postgres` to a `requirements.txt` and extending the image, or use an Airflow image variant that includes it.
- **DBeaver cannot connect:** Confirm the `postgres` container is running (`docker compose ps`) and that port `5432` is published. From the host machine always use `localhost` (not the Docker service name). If port 5432 is already in use on the host, change the published port in `docker-compose.yml` (e.g., `"5433:5432"`) and update the DBeaver connection accordingly.
- **DAG not visible in the UI:** Wait 30–60 seconds after placing the file in `dags/` for the Airflow scheduler to pick it up. Check for Python syntax errors by running `docker compose exec airflow python /opt/airflow/dags/example_postgres_dag.py`.

Getting the infrastructure right is often the hardest — and most important — part of any data engineering project. Once you can see Airflow talking to PostgreSQL and DBeaver browsing the same database from your laptop, the foundation is solid and everything built on top of it will be on stable ground!